Memory allocators in ARPEGE / AROME / IFS

A review — MPI buffers, GPU graphs, and the case for a double-stack allocator

N. Penigaud, P. Marguinaud

Météo-France

ECMWF / ACCORD

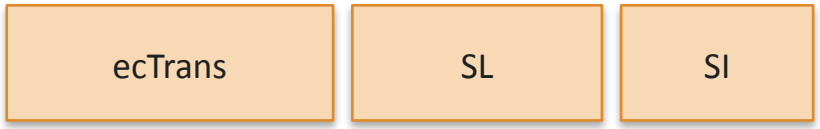
Outline

- Why a dedicated allocator for MPI and GPU-graph memory
- Constraints of each architecture: CPU vs GPU
- Methods: how to size the buffer on CPU and on GPU
- Which allocator? malloc-like, buddy, stack, double stack
- Why not a list of stacks or heaps: fragmentation
- Results on CPU and GPU, and the ecTrans code
- Conclusion and proposal

Why a dedicated allocator for MPI and graph memory?

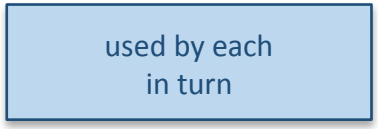
- **Fixed addresses from one time step to the next**
  - CUDA / HIP graphs are replayed without being rebuilt
  - MPI buffers can be registered (pinned) once — recommended by CPU and GPU vendors
- **No repeated ALLOCATE / DEALLOCATE**
  - one pool allocated once; each allocation is O(1) pointer arithmetic
- **One buffer shared by ecTrans, semi-Lagrangian, semi-implicit**
  - instead of one permanent buffer per component (e.g. LALLOPERM)
- **Smaller memory footprint**
  - memory is reused across components, not summed

Today: one buffer per component



memory = sum of the peaks

Proposed: one shared pool



memory = largest peak

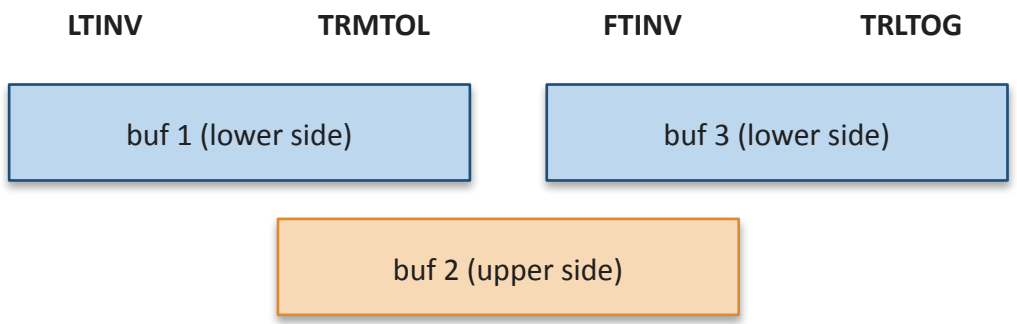
Same addresses at every step
→ graphs and MPI registrations stay valid

What we need from the allocator

The memory pattern of ecTrans, SL and SI communications

- **Local memory only**
  - buffers are allocated and freed inside the same routine
  - same behaviour as automatic arrays
- **At most two interleaved series**
  - SL, SI: one series (a plain stack is enough)
  - ecTrans: two series (input / output buffers of successive steps)
- **Deterministic order**
  - same sequence every time step → same addresses

ecTrans inverse transform (schematic)



each buffer is written by one step and read by the next: two buffers are alive at the same time, and they are freed in a different order than a single stack allows

Constraints of each architecture

CPU

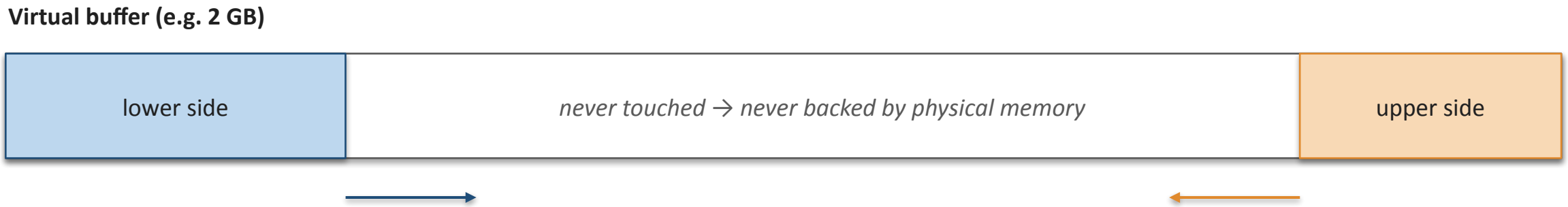
- Paged virtual memory
  - allocating a buffer only reserves addresses
  - physical pages are provided on first touch
- Over-sizing the buffer is free
  - untouched pages cost nothing (RSS)
- No need to know the size in advance

GPU

- No demand paging for device memory
  - allocated memory is physical memory
- Device memory is scarce (40–128 GB)
  - an over-sized buffer is lost for the model
- Size must be right before graphs are built
  - reallocating = new addresses = graphs rebuilt

Method on CPU: let paging do the job

- Allocate one large buffer of virtual memory, once, at start-up
  - same idea as the MPI mailbox (currently 2 GB)
- Physical memory is pulled on demand, page by page, by the operating system
- No sizing, no probe, no reallocation: addresses are fixed from the first step



RSS = pages actually touched by the two sides

Method on GPU (1): probe and reallocate

As already done for INV_TRANS

- Before each section using MPI / graph memory, a probe computes the stack size it needs
- If the current buffer is too small, reallocate it before entering the section
- INV_TRANS, the largest consumer, is already instrumented this way

Pros

- automatic: no user setting
- final size reached after the first time step

Cons

- a probe is needed in every section
- reallocations during the first time step (graphs rebuilt once)

Method on GPU (2): measure, then size

Run once with a classic allocator, record the peak, reuse it

- Make the allocator polymorphic: an allocator base class with two implementations
  - classic ALLOCATE / DEALLOCATE (measurement), and double stack (production)
- First run with the classic allocator: record the peak memory
- Subsequent runs: double stack sized with this value

Pros

- no probe in the code
- no reallocation in production runs

Cons

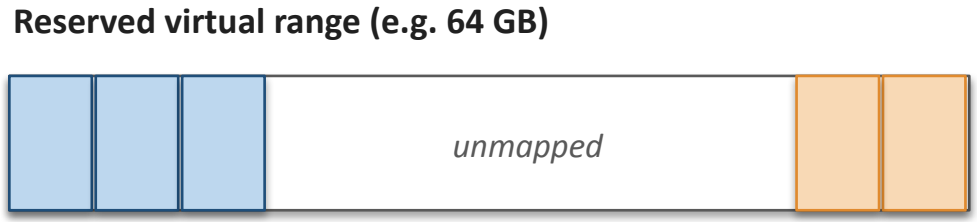
- an extra run per configuration
- the value must be stored and kept up to date

Variant: a dry run of every section without computation (measurement mode exists) — possible, but too complicated to maintain

Method on GPU (3): an idea — virtual memory on the GPU

Under investigation, not tested yet: it may not work

- CUDA and HIP separate addresses from memory
  - reserve a large virtual range: costs nothing
  - map physical chunks (~2 MB) where needed
- Double stack on top of it
  - map a new chunk when a side crosses into unmapped space
  - addresses never change: no probe, no reallocation
- The API itself is used by PyTorch and NCCL



cuMemAddressReserve / hipMemAddressReserve

cuMemCreate / hipMemCreate

cuMemMap / hipMemMap

cuMemSetAccess / hipMemSetAccess

Idea under investigation — a prototype is starting on A100, it may not work
Open questions: OpenACC / OpenMP mapping, GPU-aware MPI on this memory, AMD / ROCm support
If it fails, methods (1) and (2) remain

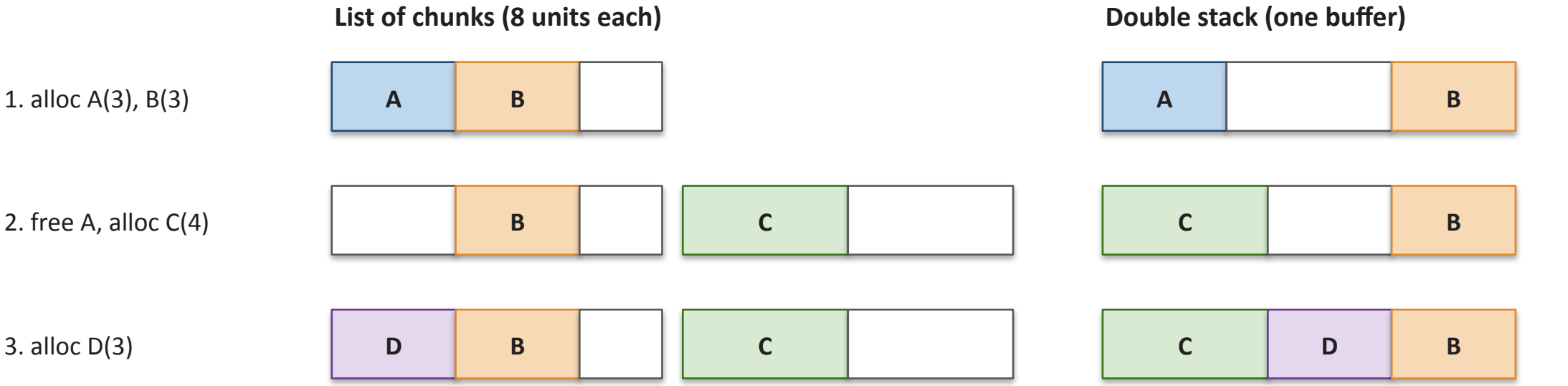
Which allocator?

	metadata apart from data	manages pure virtual memory	memory overhead	fits SL / SI	fits ecTrans
malloc-like heap	✘	✘	fragmentation	✔	✔
buddy allocator	✔	✔	up to ×2	✔	✔
stack	✔	✔	none	✔	✘
double stack	✔	✔	none	✔	✔

- malloc-like: metadata stored next to the data → touching it commits pages; unusable on a pure virtual range
- buddy (e.g. spaskalev/buddy_alloc, as in Field API): general, but rounds sizes up to powers of two
- double stack: as simple as a stack, and handles the two series of ecTrans

Why not a linked list of stacks or heaps?

Because it fragments — same sequence, two allocators



C(4) does not fit in the free 3 + 2 → new chunk
reserved 16 units for 10 live units
placement depends on history → addresses
may change from one step to the next

always compact, no holes
buffer = peak of (lower + upper) = 10 units
same sequence → same addresses
at every step

ecTrans on GPU: one object instead of a framework

Memory management in INV_TRANS and its callees (ecmwf-ifs/ectrans, main, Oct. 2026)

	today (buffered allocator)	double stack
allocator types	3	1
per-routine handle types	6	0
PREPARE_* functions (sizes computed in advance)	6	0
reservation handles	7	0
ASSIGN_PTR(GET_ALLOCATION(…)) with byte offsets	13	0
routines receiving the allocator as argument	13	0
buffer sizes computed	twice	once

With the double stack: one ALLOCATE and one DEALLOCATE per array, on a single object (MPL_DOUBLESTACK)

Results

CPU — IAL TL1798 L105, 24 h, 512 MPI × 4 threads

	STEPO (s)	RSS (MB)
double stack	419	2806
reference, LALLOPERM=T	475	3059
reference, LALLOPERM=F	485	3067

−10 % elapsed time, −8 % memory

double stack used in ecTrans and in SL / SI communications

GPU — ecTrans benchmark, 1 Hopper GPU

grid	pool	total GPU
O257	−40 %	−19 %
O539	−42 %	−19 %
O799	−42 %	−20 %
F538	−50 %	−23 %

same elapsed time (fixed addresses already in place)
e.g. O799: pool 3.97 GiB vs 6.79 GiB,
total 11.1 GiB vs 13.9 GiB

ecTrans ctest suite passes on CPU and GPU

Conclusion

- **A double-stack allocator covers our needs for ARPEGE, AROME and IFS**
  - MPI buffers and GPU-graph memory, in ecTrans, SL and SI
- **It is simple: easy to implement, easy to read**
  - already implemented in FIAT, ecTrans and IAL
- **CPU: −10 % elapsed time, −8 % memory**
- **GPU: smaller footprint, and a much simpler ecTrans code**
- **One decision remains for the GPU: how to size the pool**
  - probe and reallocate, or measure then size; GPU virtual memory is an idea under investigation

Our proposal

- We evaluated the alternatives: heaps, buddy allocators, stacks, lists of chunks
- The double stack is simple, implemented on CPU and GPU, and measured
- We propose to adopt it in FIAT, ecTrans and IAL

Counter-proposals are welcome —
with measurements on the same test cases